A computer-verified lower bound for the ninth term of OEIS sequence A338700

Carlo Corti
Independent researcher
carlo.corti@outlook.com

May 4, 2026

Abstract

The OEIS sequence A338700 records, for each $n \geq 1$, the smallest prime p_k such that n consecutive primes $p_k, p_{k+1}, \ldots, p_{k+n-1}$ are all Sophie Germain primes. Eight terms are known, the largest being $a(8) = 1\,372\,604\,395\,439$ (Ehrenstein, 2021). We report an exhaustive computer search of the interval $[a(8), 5 \cdot 10^{14}]$ that did not yield any new term. Consequently we establish the verified lower bound

$$a(9) > 5 \cdot 10^{14}.$$

The search processed 15 186 821 470 495 candidate primes over a total wall-clock time of 52 h 49 min on a 16-thread consumer-class machine (AMD Ryzen 9 7940HS). The source code, selected campaign log, and reproducibility artefacts are released under the MIT licence [9].

1 Definition and known terms

Let $p_1 < p_2 < p_3 < \ldots$ denote the primes in increasing order. A prime p is called a *Sophie Germain prime* if $2p + 1$ is also prime; the set of such primes is OEIS A005384 [2]. Long runs of consecutive Sophie Germain primes are rare and their distribution is governed by a Hardy–Littlewood type heuristic [4] (§3). Each new term of A338700 represents a record-length run, and exact computation of $a(9)$ would be the first extension of the sequence since Ehrenstein's 2021 contribution of $a(8)$. The present work does not find $a(9)$ but establishes a substantially improved lower bound.

Definition 1 (OEIS A338700 [1])**.** For $n \geq 1$, $a(n)$ is the smallest prime p_k such that the n consecutive primes $p_k, p_{k+1}, \ldots, p_{k+n-1}$ are all Sophie Germain primes.

The function $n \mapsto a(n)$ is non-decreasing: any run of length $n + 1$ contains a run of length n at the same starting prime, hence $a(n + 1) \geq a(n)$, with equality possible if the first run of length n already happens to extend to length $n + 1$.

The eight known terms are listed in Table 1.

2 Search strategy

2.1 Reduction to a sieve problem

Finding $a(n)$ for a given n amounts to scanning the primes in increasing order and maintaining the length of the current run of consecutive Sophie Germain primes; the first index at which the run reaches length n yields $a(n)$. Two equivalent computational kernels are available:

Table 1: Known terms of A338700.

n	$a(n)$	Source
1	2	trivial
2	2	trivial
3	2	trivial
4	1 433 849	OEIS
5	9 816 899	OEIS
6	445 480 319	OEIS
7	298 098 924 131	G. Resta, 2004 [3]
8	1 372 604 395 439	M. Ehrenstein, 2021 [1]

- **MR mode.** Generate the primes p in $[L, U]$ with a segmented sieve of Eratosthenes, then test each p for the Sophie Germain property by a Miller–Rabin primality test [5, 6] on $2p + 1$.
- **Double-sieve mode.** Sieve simultaneously the interval $p \in [L, U]$ (yielding a bitmap P of primes) and the interval $q \in [2L + 1,\ 2U + 1]$ (yielding a bitmap Q of primes). The Sophie Germain primes in $[L, U]$ are then exactly the indices for which both $P[p]$ and $Q[2p + 1]$ are set. No Miller–Rabin call is needed.

For the range of interest the double-sieve mode is faster because Miller–Rabin on a 64-bit operand costs several hundred CPU cycles, whereas a sieve strike is a single bit operation. The production campaign used double-sieve mode throughout.

Both bitmaps are sieved using base primes up to $\lceil \sqrt{2U + 1} \rceil$. For the production campaign ($U = 5 \cdot 10^{14}$) this bound is approximately $3.16 \cdot 10^7$; the resulting list fits in roughly 14 MB and is shared read-only across all threads.

2.2 Numerical types

For $p < 2^{63}$ one has $2p + 1 < 2^{64}$, so all prime arithmetic fits in `uint64_t`. Modular multiplications inside Miller–Rabin (retained for the validation harness) use `__uint128_t` intermediate products, which GCC compiles to native `MUL/DIV` instructions on x86-64 without library calls.

The Miller–Rabin implementation is *deterministic* for all inputs below $3.317 \cdot 10^{24}$ using the twelve witness bases $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$, which covers the full range $n < 2^{64}$ relevant to this search. This deterministic bound is established in [7].

2.3 Algorithm and block-boundary reconciliation

The interval $[L, U]$ is partitioned into contiguous *blocks* of fixed width $W = 2^{21}$ odd integers (i.e. each block $[\ell_i, h_i]$ spans W consecutive odd integers). Within each block, the per-block kernel (Algorithm 1) sieves both P and Q, walks the primes in $[\ell_i, h_i]$ in increasing order, and emits a fixed-size summary s_i together with all runs entirely contained in the block. After the parallel phase completes, a deterministic serial pass (Algorithm 2) walks the summaries $s_0, s_1, \ldots$ in order and reconstructs the runs that cross block boundaries.

Two integer parameters govern run tracking throughout:

- $n_{\min}$ (`cfg_target`): the target run length, equal to 9 for the production campaign. A run is recorded only when its length reaches or exceeds $n_{\min}$.
- M (`max_run`): the run-length cap, set equal to $n_{\min}$ in the production campaign. Runs longer than M are truncated to M in the carry and prefix/suffix fields; since any run of length $\geq n_{\min}$ disproves the lower bound, this truncation does not affect the correctness of the search.

Per-block summary. Each block emits a record s_i with the following fields, where "Sophie Germain prime" is abbreviated SG:

- `num_primes`: the number of primes in $[\ell_i, h_i]$;
- `first_prime`, `last_prime`: the smallest and largest primes in $[\ell_i, h_i]$ (or 0 if the block contains no primes);
- `prefix_len`, `prefix_start`: length and starting prime of the maximal SG-run that begins at `first_prime`, or 0 if `first_prime` is not SG;
- `suffix_len`, `suffix_start`: length and starting prime of the maximal SG-run that ends at `last_prime`, or 0 if `last_prime` is not SG;
- `all_sg`: a Boolean flag, true iff every prime in the block is SG (computed directly in Algorithm 1, line 4 and updated through the loop).

Algorithm 1 Per-block scan (executed once per block, in parallel)

Require: base primes, block range $[\ell_i, h_i]$, run-length cap M, recording threshold $n_{\min}$

1: sieve P on odd integers in $[\ell_i, h_i]$
2: sieve Q on odd integers in $[2\ell_i + 1, 2h_i + 1]$
3: $run \leftarrow 0$; $run\_start \leftarrow 0$
4: zero all fields of s_i; s_i.`all_sg` $\leftarrow$ **true**
5: **for all** primes p in $[\ell_i, h_i]$ taken in increasing order **do**
6: s_i.`num_primes` += 1
7: **if** s_i.`first_prime` $= 0$ **then** s_i.`first_prime` $\leftarrow p$
8: **end if**
9: s_i.`last_prime` $\leftarrow p$
10: $sg \leftarrow P[p] \wedge Q[2p + 1]$
11: **if** sg **then**
12: **if** $run = 0$ **then** $run\_start \leftarrow p$
13: **end if**
14: $run \leftarrow \min(run + 1, M)$
15: **if** $run \geq n_{\min}$ **then**
16: record run of length run at run_start in thread-local record
17: **end if**
18: **else**
19: s_i.`all_sg` $\leftarrow$ **false**
20: $run \leftarrow 0$
21: **end if**
22: maintain s_i.`prefix_*` until first non-SG prime; maintain s_i.`suffix_*` at every SG-extending step
23: **end for**

Invariant and correctness. Let $r(p)$ denote the length of the maximal run of consecutive SG primes ending at the prime p, taken over the global prime sequence (not restricted to any block). The reconciliation algorithm maintains the following invariant.

Proposition 1 (Reconciliation invariant)**.** *After processing summaries $s_0, \ldots, s_{i-1}$:*

- *if the last prime processed so far is SG, then carry equals* $\min(r(\textit{last prime}), M)$ *and carry_start is the starting prime of that run;*

Algorithm 2 Boundary reconciliation (executed serially after the parallel phase)

Require: summaries $s_0, s_1, \ldots, s_{B-1}$ in block order; cap M; recording threshold $n_{\min}$

1: $carry \leftarrow 0$; $carry\_start \leftarrow 0$
2: **for** $i = 0, 1, \ldots, B-1$ **do**
3: **if** s_i.`num_primes` $= 0$ **then continue**
4: **end if**
5: **if** $carry > 0 \wedge s_i$.`prefix_len` > 0 **then**
6: $merged \leftarrow \min(carry + s_i$.`prefix_len`$, M)$
7: **if** $merged \geq n_{\min}$ **then**
8: record run of length $merged$ at $carry\_start$ in global record
9: **end if**
10: **end if**
11: **if** s_i.`all_sg` **then**
12: **if** $carry = 0$ **then** $carry\_start \leftarrow s_i$.`first_prime`
13: **end if**
14: $carry \leftarrow \min(carry + s_i$.`num_primes`$, M)$
15: **if** $carry \geq n_{\min}$ **then**
16: record run of length $carry$ at $carry\_start$ in global record ▷ multi-block run
17: **end if**
18: **else if** s_i.`suffix_len` > 0 **then**
19: $carry \leftarrow s_i$.`suffix_len`; $carry\_start \leftarrow s_i$.`suffix_start`
20: **else**
21: $carry \leftarrow 0$; $carry\_start \leftarrow 0$
22: **end if**
23: **end for**

- *otherwise* $carry = 0$.

Moreover, every maximal SG-run that ends within $\bigcup_{j<i}[\ell_j, h_j]$ *has been recorded in either a thread-local record (Algorithm 1) or the global record (Algorithm 2), at least once. (For the purposes of this search, recording a run multiple times is harmless: the global record retains only the best starting prime per run length.)*

Empirical verification (sketch). The invariant is not formally proved by induction in this manuscript; correctness is established empirically through the three-level validation harness (§2.4) and the overlapping-window cross-tests. The following sketch explains the design intent. A maximal SG-run either lies entirely within a single block (recorded by Algorithm 1, line 14) or crosses one or more block boundaries. In the latter case, its starting prime lies either at the first prime of some block (captured by s_i.`prefix_*`) or at s_i.`suffix_start` for the leftmost block i in which the run is non-empty. Algorithm 2 closes the cross-boundary run exactly when its right endpoint occurs (line 5), and never closes the same run twice because *carry* is reset on the first non-SG prime encountered afterwards. Multi-block all-SG runs are tracked by accumulating *carry* across consecutive all-SG blocks (line 12).

Concurrency discipline. The parallel phase has no shared mutable state. Each thread t writes exclusively to (i) its own block summary s_i (the assignment $i \mapsto t$ is fixed by the OpenMP schedule for the duration of that block), and (ii) its own record buffer `thread_rec`$[t]$. Both structures are cache-line aligned (64 bytes) to prevent false sharing. After the parallel phase, the serial post-pass merges all `thread_rec`$[t]$ into a global record and then runs Algorithm 2. There are no locks, atomics, or barriers within the parallel phase other than the implicit end-of-region barrier.

2.4 Validation harness

Three validation levels were used to certify the implementation before the production campaign:

- `small`: $[0, 10^7]$, verifies $a(1), \ldots, a(5)$ (under one second);
- `medium`: $[0, 5 \cdot 10^8]$, verifies $a(1), \ldots, a(6)$ (seconds);
- `full`: $[0, 2 \cdot 10^{12}]$, verifies $a(1), \ldots, a(8)$ (hours).

All three levels reproduce the corresponding OEIS values exactly in both MR and double-sieve mode; the agreement is guaranteed (not merely probabilistic) because the MR harness uses the twelve-base deterministic witness set described in §2.2. Block-boundary stress tests (eight overlapping windows, both modes, each tested against the other) all match byte-for-byte.

The double-sieve operates on odd integers only, so the prime $p = 2$ is handled as a special case in validation mode: when the search interval includes 2 (i.e. $L \leq 2$), the harness explicitly initialises the run-tracking state to reflect $p = 2$ as the first Sophie Germain prime ($2p + 1 = 5$ is prime) before entering the odd-only sieve at $p = 3$. This special case applies only to the validation tiers; it is not exercised by the production campaign, whose lower bound $L = a(8)$ is odd.

3 The campaign

3.1 Search window

Because A338700 is non-decreasing, $a(9) \geq a(8)$ and the search may start at $L = a(8) = 1\,372\,604\,395\,439$. Equality is possible (the run of length 8 starting at $a(8)$ might extend to length 9 or more), so the implementation includes the starting prime $a(8)$ in its scan. The initial carry is set to 0: the run-length analysis is self-contained within $[L, U]$ and does not depend on primes below L.

The upper bound $U = 5 \cdot 10^{14}$ was chosen as a round value approximately 360 times larger than $a(8)$, reachable within a reasonable wall-clock budget on the available hardware. Under the Hardy–Littlewood heuristic [4] the median prediction for $a(9)$ is of order 10^{14}, so U exceeds it by a factor of ~ 5. With $L \approx 1.37 \cdot 10^{12}$ this gives a search interval of width $\sim 4.99 \cdot 10^{14}$, of order 10^{14}.

3.2 Hardware and software

- CPU: AMD Ryzen 9 7940HS (Zen 4, 8 cores / 16 threads).
- RAM: 64 GB DDR5.
- OS: Linux Mint 22.3.
- Compiler: GCC 13.3.0, C17, flags `-O2 -march=native -fopenmp -std=c17`.
- OpenMP schedule: `OMP_SCHEDULE=guided,4`.
- Implementation: 16 threads, 16-way parallel double-sieve, segment of 2 097 152 odd candidates per block, batches of 4 096 blocks each.
- Licence: MIT.

The full source code, build system, validation harness, and selected campaign log are released in the repository [9].

3.3 Results

The campaign was carried out in ten consecutive sessions, with periodic checkpoints to a state file allowing exact resume. Aggregate figures:

Table 2: Campaign aggregates.

Search window	$[1\,372\,604\,395\,439,\ 5 \cdot 10^{14}]$
Total wall-clock time (10 sessions)	52 h 49 min 11 s
Total batches processed	29 024
Primes scanned	15 186 821 470 495
Peak throughput (warm batch, 16 threads)	$\sim 1.76 \cdot 10^9$ odd cand./s
Average throughput (full campaign)	$\sim 8.0 \cdot 10^7$ primes/s
Sophie Germain primes found in window	598 620 723 232
Longest run found in window	length 8, at $p = a(8)$
New term $a(9)$	not found

The peak figure refers to the throughput of odd-integer candidates processed by the double-sieve in the warmest batches of the final session; it is measured in sieve slots per second, not confirmed primes per second. The average throughput is computed from total primes scanned divided by total wall-clock time (190 151 s) and reflects the overhead of ten session starts, periodic checkpoint writes during execution, and the natural variation in prime density across the interval.

The proportion of Sophie Germain primes among the primes in the window,

$$\frac{598\,620\,723\,232}{15\,186\,821\,470\,495} \approx 0.0394,$$

is consistent with the Hardy–Littlewood prediction [4] $\pi_{\mathrm{SG}}(x)/\pi(x) \sim 2C_2/\log x$ with twin-prime constant $C_2 \approx 0.6601618$ at the relevant scale $\log x \approx 33$, which gives the heuristic value ≈ 0.0400.

4 Result

The exhaustive search produced the following verified statement.

Theorem 2. *Let $a(n)$ denote the n-th term of OEIS A338700. Then*

$$a(9) > 5 \cdot 10^{14}.$$

Verification. Every prime in the closed interval $[a(8),\ 5 \cdot 10^{14}]$ was classified as Sophie Germain or not, by means of the double-sieve procedure described in §2. The longest run of consecutive Sophie Germain primes recorded in this interval has length 8 and starts at $p = a(8)$; no run of length 9 exists in the interval. The implementation reproduces $a(1), \ldots, a(8)$ exactly under three independent validation levels including a full re-derivation up to $2 \cdot 10^{12}$. Block-boundary reconciliation is verified by overlapping-window cross-tests and by equivalence between two independent kernels (MR and double-sieve). The selected campaign log and source code are archived at [9]. The lower bound has been communicated to the OEIS and is recorded at [8]. □

5 Reproducibility

The repository [9] contains:

- the full C17 source (single binary, < 3000 LOC);

- a `Makefile` with five main targets: `validate-small`, `validate-medium`, `validate-full`, `bench-schedule`, `release-znver4`;
- the selected campaign log (`a338700_campaign.log`), documenting the main campaign sessions.

The exact compilation flags, OpenMP schedule, and block parameters used in the production binary are recorded in the repository README.

5.1 Edge-block handling

The search interval $[L, U]$ is padded internally to a multiple of `block_odds` $= 2^{21}$ odd integers. Positions beyond U in the final block are masked out before the per-block scan emits its summary, so that all summary fields refer only to primes in $[L, U]$. The sieve itself does no bounds-checking on the padded region; correctness rests on the masking step.

5.2 Checkpoint format and resume integrity

The checkpoint file holds a single `state_t` structure with the following fields, in order:

- a 32-bit magic number and a 32-bit format version (currently 2);
- the search configuration: `cfg_low`, `cfg_high`, `cfg_block_odds`, `cfg_batch_blocks`, `cfg_target`, `cfg_max_run`, `cfg_mr`;
- the next-batch index;
- the carry pair (*carry*, *carry_start*) from Proposition 1;
- the running totals `total_primes` and `total_sg`;
- the global record `global_rec` (best run-starting primes per length);
- a 64-bit checksum.

The checksum is a byte-wise 8-lane XOR over the structure up to (but excluding) the checksum field itself. On resume, the file is rejected if the magic, version, or checksum does not match. Configuration fields loaded from the file overwrite the runtime configuration, so the resumed run uses exactly the same search parameters as the interrupted one. Writes are atomic: the new state is written to a temporary file, fsynced, and renamed over the previous state file, after which the parent directory is fsynced as well.

AI use disclosure

This work was carried out with substantial support from large language models, used as a research assistant throughout the full workflow. Specifically: mathematical analysis of the candidate space and Hardy–Littlewood heuristics (Claude Opus 4.7, Anthropic); design of the computational strategy (Claude Opus 4.7); code generation in C17 (GPT-5.5, OpenAI); LLM-based code review of successive versions, with six independent model passes (Claude, ChatGPT, DeepSeek, Gemini, Grok, Qwen); and composition of this manuscript (Claude Sonnet 4.6, Anthropic). Every quantitative claim in this paper was verified computationally by the author; all mathematical statements were checked by the author before inclusion. The author takes full scientific responsibility for the content. The exact model versions and API identifiers used at each stage are recorded in the repository at [9].

References

[1] The OEIS Foundation Inc., *Sequence A338700: Smallest prime p such that p and the $n-1$ following primes are all Sophie Germain primes*, The On-Line Encyclopedia of Integer Sequences, https://oeis.org/A338700. Sequence created by M. Sariyar, April 24, 2021; term $a(8)$ contributed by M. Ehrenstein, April 29, 2021.

[2] The OEIS Foundation Inc., *Sequence A005384: Sophie Germain primes p: $2p+1$ is also prime*, The On-Line Encyclopedia of Integer Sequences, https://oeis.org/A005384.

[3] C. Rivera, *Puzzle 71: Consecutive primes and Cunningham chains*, The Prime Puzzles & Problems Connection, https://www.primepuzzles.net/puzzles/puzz_071.htm.

[4] G. H. Hardy and J. E. Littlewood, *Some problems of 'Partitio Numerorum' III: On the expression of a number as a sum of primes*, Acta Mathematica **44** (1923), 1–70. DOI: 10.1007/BF02403921.

[5] G. L. Miller, *Riemann's hypothesis and tests for primality*, Journal of Computer and System Sciences **13** (1976), 300–317. DOI: 10.1016/S0022-0000(76)80043-8.

[6] M. O. Rabin, *Probabilistic algorithm for testing primality*, Journal of Number Theory **12** (1980), 128–138. DOI: 10.1016/0022-314X(80)90084-0.

[7] J. Sorenson and J. Webster, *Strong pseudoprimes to twelve prime bases*, Mathematics of Computation **86** (2017), 985–1003. DOI: 10.1090/mcom/3134. The twelve bases $\{2,3,5,7,11,13,17,19,23,29,31,37\}$ yield a deterministic test for all $n < 3.317 \cdot 10^{24}$, covering the full `uint64_t` range used in this work.

[8] C. Corti, *Contribution to OEIS A338700: lower bound $a(9) > 5 \cdot 10^{14}$*, The On-Line Encyclopedia of Integer Sequences, https://oeis.org/A338700, 2026.

[9] C. Corti, *A338700 — extending the OEIS sequence: source code, logs and reproducibility artefacts*, GitHub repository: https://github.com/carcorti/A338700. Archived release: Zenodo, DOI 10.5281/zenodo.20039591.